



A distributed storage MLCS algorithm with time efficient upper bound and precise lower bound

Chunyang Wang^a, Yuping Wang^{a,*}, Xiangjuan Wu^a, Xiaofang Guo^b

^a School of Computer Science and Technology, Xidian University, Xian, Shaanxi, China

^b School of Science, Xian Technological University, Xian, Shaanxi, China

ARTICLE INFO

Article history:

Received 15 February 2022

Received in revised form 8 September 2022

Accepted 26 October 2022

Available online 1 November 2022

Keywords:

Multiple longest common subsequences problem

Distributed storage

Biological data mining

ABSTRACT

Finding the longest common subsequence of multiple sequences (i.e., the MLCS problem) is a key issue in data mining, such as pattern recognition of bio-sequences (e.g., DNA and protein sequences), diagnosis of diseases, and deciphering human genetic codes. The length of the longest common subsequence is often used to measure the similarity between sequences. Current state-of-the-art algorithms for the MLCS problem often transform the problem into finding the longest path in a directed acyclic graph (DAG); however, their fatal flaw is that the time cost or constructed DAG is too large. Aiming at solving the problem of the existing algorithms, this paper proposes a time-efficient upper bound estimation method that can estimate the upper bound for the length of any path through a node in much less time, and then proposes an adaptive lower bound estimation strategy that can estimate the more precise lower bound for the length of the longest common subsequences. This study also describes a distributed storage scheme that stores most parts of the DAG in external devices and only a few parts in memory. A comparison with several state-of-the-art algorithms in the experiments indicates that the proposed algorithm is more efficient and effective.

© 2022 Elsevier Inc. All rights reserved.

1. Introduction

In the Human Genome Project, approximately three billion base pairs of DNA and protein sequences have been determined. In order to decipher the genetic code contained in biological sequences for disease diagnosis and treatment [1], protein prediction [2,3], genetic engineering [4–6], and many other application aspects [7,8], countless scholars have studied genetic data mining. Identifying the longest common subsequence of multiple sequences (MLCS) for DNA or protein sequences is an effective technique for developing new disease diagnosis methods. If only two sequences exist, the problem can be solved perfectly using the dynamic programming method [9]. When the number of sequences is large, the dynamic programming method is ineffective. The most effective methods are those that convert the MLCS problem into a longest path problem in a directed acyclic graph (DAG) (e.g., [10–15]). Among them, Top-MLCS [14] is a new method for constructing a much smaller DAG without redundant nodes (called an irredundant common subsequence graph, ICSG) than those built by Fast-MLCS [11], Quick-DPPAR [13], and non-dominated tree strategy [12]. The ICSG model uses hash table technology to avoid both the appearance of duplicated nodes in the graph and the non-dominated sorting of high-dimensional points. Thus, the space

* Corresponding author.

E-mail addresses: cywang1996@stu.xidian.edu.cn (C. Wang), ywang@xidian.edu.cn (Y. Wang), xjwu_2017@stu.xidian.edu.cn (X. Wu), guoxiaofang@xatu.edu.cn (X. Guo).

and computation costs are significantly decreased. However, for large-scale MLCS problems, the number of nodes stored in the hash table will be too large, and no tailor-made strategy exists to reduce the storage, resulting in a lot of time and storage wasted during ICSG construction. Furthermore, the algorithm needs to use two topological sorting schemes, which also require an additional time overhead. In 2019, Liu et al. [16] proposed a character merging algorithm (CMA). It merges repeated characters in the sequences and builds a weighted DAG model, which is much smaller than the DAG built by the Top-MLCS algorithm. However, when there are few repeated characters in the sequences, the improvement in the CMA is not sufficiently significant. In 2020, Wei et al. [17] proposed the PRA-MLCS algorithm. It uses a new data structure to record the constructed DAG, which reduces the time required for topological sorting; however, the scale of the DAG does not decrease. Therefore, it is still inefficient when facing large-scale MLCS problems. To overcome these shortcomings, Big-MLCS [15] proposes a branch-and-bound method to determine whether a node is on the longest path of the DAG. It uses two heuristic methods to estimate a lower bound for the length of the longest common subsequences, and an upper bound for the length of any path through the current node. If the upper bound is smaller than the lower bound, then no path through the current node will be the longest path. It is not required to store this node in the DAG (thus, saving storage and computation). Thus, the Big-MLCS requires a much smaller space and time to construct the DAG and look for all MLCSs. However, it still has the following drawbacks:

- Note that utilizing efficiently calculable upper-bound calculation is one of the key schemes for a branch-and-bound method, but the estimation scheme of the upper bound in Big-MLCS is not very efficient. It needs to compute the difference between the shortest sequence S_i and each of the remaining sequences, leading to too much time consumption.
- Note that estimating a precise lower bound is also one of the key schemes for a branch-and-bound method, but the estimation scheme of the lower bound in Big-MLCS is inefficient. It only gives an initial estimation at the beginning and shows no adaptive improvement for more precise estimations. The adaptively improved estimations can reduce space and time costs greatly.
- Although the DAG constructed by Big-MLCS is much smaller than those built by Top-MLCS and other state-of-the-art methods, it is still very large for large-scale problems. Big-MLCS needs to store the entire built DAG into memory. This still occupies a large amount of memory and results in memory overflow for large-scale MLCS problems.

The aforementioned methods are exact MLCS algorithms that look for all true MLCSs. Another type of method is the approximate MLCS method, which attempts to look for an approximate MLCS as quickly as possible (e.g., Pro-MLCS [18], SA-MLCS [19], and SLA-MLCS [19]). A^* algorithm proposed recently [20,21] is a typical example of such algorithms.

In this study, we investigated the exact MLCS methods. To overcome the aforementioned shortcomings of exact MLCS methods, we propose a new method that uses the following new techniques:

- **Propose a time efficient upper bound estimation method.** We propose a new method to estimate the upper bound of any path through the current node. The new method uses much less time, but is still high quality, making it more efficient.
- **Design a precise lower bound estimation method.** Differently from the fixed lower bound provided by the existing methods, an adaptively updated lower bound estimation method is proposed. The estimated lower bound becomes increasingly precise during the construction of DAG. As a result, many nodes will not be stored in DAG by utilizing this lower bound.
- **Propose an efficient distributed storage method which only needs to store a small part of DAG in the memory and a large part in external storage devices in a distributed way at any moment.** Storing the entire built DAG can result in the memory overflow and is a key drawback of existing methods because of the large size of the entire DAG. To avoid doing this, we propose a new storage method which only stores a small part of DAG in memory and distributes the remaining part in different external storage devices. In this way, we can save a lot of memory, improve the efficiency of the method, and handle much larger scale problems.

The remainder of this paper is organized as follows. Section 2 introduces some preliminaries, including symbols, concepts, and related work. In Section 3, the proposed algorithm is described. Section 4 verifies the effectiveness of the proposed algorithm experimentally. Finally, in Section 5, we draw conclusions and provide ideas for future work.

2. Preliminary Definitions and Related Works

In this section, we will define some symbols and concepts, then introduce the main contributions of two state-of-the-art exact algorithms, Top-MLCS and Big-MLCS, and finally introduce algorithms based on the A^* framework.

2.1. Symbols and Concepts

Sequence and Alphabet: A sequence S denotes a sequence of characters from the alphabet Σ . For example, a biological sequence (e.g., a DNA or protein sequence) is a sequence of characters from $\Sigma = \{C, G, A, T\}$ or from a set of 20 amino acids, respectively.

Multiple Longest Common Subsequences (MLCS) Problem: S' is called a subsequence of S if S' is obtained by deleting some character(s) or by not deleting any characters from S . For n sequences ($n \geq 3$), if S' is a subsequence of all of these n sequences, it is called a common subsequence of them. If S' is the longest one among the common subsequences, it is called the longest common subsequence (usually there is more than one such S'). The set of these S' is called the multiple longest common subsequence set, and the problem of finding this set is called the multiple longest common subsequence problem (MLCS problem).

Match Point and Dominant Point: For a sequence set $S = \{S_1, \dots, S_i, \dots, S_n\}$, let $S_i[j]$ represent the j -th character of sequence S_i ($1 \leq i \leq n$). If $S_1[p_1] = \dots = S_n[p_n] = \alpha \in \Sigma$, we call $P = (p_1, p_2, \dots, p_n)$ a match point of the character α (briefly, a match point). For two match points, $P = (p_1, p_2, \dots, p_n)$ and $Q = (q_1, q_2, \dots, q_n)$, if they satisfy $\forall i (1 \leq i \leq n), p_i < q_i$, then we say that P dominates Q (denoted as $P \prec Q$). For a set M_s of match points and $P \in M_s$, if no point $Q \in M_s$ dominates P , then P is called a dominant point on M_s .

Successor and Immediate Successor: If match points P and Q satisfy $P \prec Q$, then we call Q the successor of P or P the predecessor of Q . If no other successor exists between P and Q (i.e., $\nexists R$ satisfying $P \prec R \prec Q$), then Q is called the immediate successor of P .

In-degree of a Node: For a node in a graph, the number of edges pointing to it from other nodes is called the in-degree.

2.2. Top-MLCS Algorithm[14]

The framework of the top-MLCS algorithm includes three main steps:

1. Construct ICSG.
2. Determine Levels of Nodes by Forward Topological Sorting.
3. Backward Topological Sorting to Get the Final MLCS.

For the detailed procedure of the Top-MLCS algorithm, please refer to Appendix A.

The main defects of the Top-MLCS algorithm are as follows: 1) It requires considerable storage space to record the nodes in the hash table. 2) To obtain the final result, it requires two topological sorting operations with a lot of computation costs. 3) It requires storage of the entire ICSG in the memory. 4) The scale of ICSG is too large for large-scale problems, resulting in memory overflow.

2.3. Big-MLCS Algorithm[15]

To better solve the large-scale MLCS problem, Wang et al. [15] proposed the Big-MLCS algorithm and designed a branch-and-bound scheme to reduce the scale of the DAG. The scheme first estimates a lower bound for the MLCS by a heuristic search, that is, it looks for a specific path that is as long as possible and uses its length as a lower bound for the length of the MLCS. Second, it designs a key node selection scheme at the current level. Third, it estimates an upper bound for the path length from the start node to the end node through the key node. If the upper bound is smaller than the estimated lower bound, no path through the key node is the longest, and it is unnecessary to further construct the later paths from the key node.

We explain the details of these strategies using the following example with three sequences:

$S_1 = ACAGTT$
 $S_2 = CAGGCT$
 $S_3 = GACAGTA$

2.3.1. Estimate A Lower Bound of MLCS L_{MLCS} .

The main idea is as follows: Find a specific path in the DAG that is as long as possible and use its length as a lower bound of the MLCS. From the first level, select a node $P(p_1, p_2, \dots, p_n)$ with the smallest *sum* of its components at each level.

$$sum = \sum_{i=1}^n p_i \quad (1)$$

If more than one node attains the minimum, select the node with the smallest *dis*:

$$dis = \sum_{i=1}^n \left| p_i - \frac{sum}{n} \right| \quad (2)$$

If more than one node attains the smallest *sum* and smallest *dis* simultaneously, one node is randomly selected from among them.

For the example above, we choose $A(1, 2, 2)$ in the first level because the sum of its components is the smallest. This process was repeated until a real path was formed, as shown in Fig. 1. The length of this path is three and is used as the lower bound $L_{MLCS} = 3$.

2.3.2. Select The Key Node P in Current Level.

At each level from level 1, the node with the minimum component is found. If there are multiple nodes with the same minimum component, then one is randomly selected $P(p_1, p_2, \dots, p_n)$ as a key node.

In the above example, we can find that nodes $A(1, 2, 2)$, $C(2, 1, 3)$, and $G(4, 3, 1)$ in level 1 have minimum component 1, so we can randomly select one, for example, $A(1, 2, 2)$, as a key node.

2.3.3. Estimate An Upper Bound through Key Node:

$U(O, P, \infty)$ represents an upper bound of the length for the path from the start node O to the end node ∞ through the key node $P(p_1, p_2, \dots, p_n)$. This upper bound is divided into two parts: the length of the longest path from the start node to the key node, denoted by $Level(P)$, and the upper bound of the length of any path from the key node to the end node, denoted by $U(P, \infty)$. The first part is the level of the key node and is known. The estimation for the second part is as follows:

For key node $P(p_1, p_2, \dots, p_n)$, consider the subsequences $\{S_1^p, S_2^p, \dots, S_n^p\}$ after node P from the original sequences $S_1, S_2, \dots, S_n$.

First, we select the shortest subsequence and denote it as S_* . We denote this length as $lenS_*$. Let N_S^α denote the number of characters α in the subsequence S . Then, the diversity between S_* and any other subsequence S_i^p is calculated as follows:

$$diversity(S_*, S_i^p) = \sum_{\alpha \in \Sigma} \frac{N_{S_*}^\alpha}{lenS_*} (N_{S_*}^\alpha - N_{S_i^p}^\alpha) \quad (3)$$

Then, S_k^p with the largest diversity to S_* is selected. Finally, the length of the longest common subsequence (LCS) between S_* and S_k^p is considered as $U(P, \infty)$. An example of estimating an upper bound through a key node and the main steps of Big-MLCS are given in Appendix B.

Even though Big-MLCS has a great improvement in time and space consumption compared with the existing algorithms, including Top-MLCS, it still has the following problems:

- When estimating the upper bound $U(P, \infty)$, all sequences except S_* are considered when selecting the second sequence, which leads to excessive time consumption in calculating diversity.
- The lower bound L_{MLCS} is estimated only once and there is no updating strategy, but a more and more precise lower bound estimation is possible by updation and it can reduce the time and space overhead.
- The entire DAG is stored in memory and this consumes a lot of space. For large-scale MLCS problems, storage of DAG can still exceed the memory limit of the computer.

2.4. Algorithms Based on The A^* Framework

The core idea of algorithms based on the A^* framework is to maintain a priority queue for nodes, where a node P with a larger upper bound $U(O, P, \infty)$ has a higher priority. Each time, the node with the maximum $U(O, P, \infty)$ will be expanded first. The upper bound comprises two parts. The first part is the length of the longest path from the start node to node P and the second part is the upper bound of the length of any path from P to the end node. It is typically the best one selected from more than one upper bounds. For example, Djukanovic et al. [20] provided two different upper bounds, denoted as $U_1(P, \infty)$ and $U_2(P, \infty)$. $U_1(P, \infty)$ is calculated as the sum of the minimum number of characters α in all subsequences $\{S_1^p, S_2^p, \dots, S_n^p\}$ after node P .

$$U_1(P, \infty) = \sum_{\alpha \in \Sigma} \min_{i=1, \dots, n} N_{S_i^p}^\alpha \quad (4)$$

$U_1(P, \infty)$ can be computed with the complexity $O(|\Sigma|n)$.

After obtaining the length $LCS(S_i^p, S_{i+1}^p)$ of the LCS of S_i^p and S_{i+1}^p in preprocessing for $i = 1, 2, \dots, n-1$, $U_2(P, \infty)$ can be obtained by

$$U_2(P, \infty) = \min_{i=1, \dots, n-1} LCS(S_i^p, S_{i+1}^p) \quad (5)$$

with complexity $O(n)$. Finally, $U(P, \infty) = \min\{U_1(P, \infty), U_2(P, \infty)\}$, and the complexity of computing $U(P, \infty)$ is $O(|\Sigma|n) + O(n) = O(|\Sigma|n)$.

Recently, Horn et al. [21] proposed a new upper bound estimation method that chooses the best one from three estimated upper bounds $U_1(P, \infty)$, $U_2(P, \infty)$, and $U_3(P, \infty)$, where $U_1(P, \infty)$ and $U_2(P, \infty)$ are defined above, and $U_3(P, \infty)$ is computed as follows:

First, for each pair of input sequences $S_i, S_j \in S$, we calculate $M(S_i, S_j)$ as

$$M(S_i, S_j) = \sum_{\alpha \in \Sigma} \min(N_{S_i}^\alpha, N_{S_j}^\alpha) \quad (6)$$

for $1 \leq i < j \leq n$.

Second, n pairs of sequences (S_i, S_j) with the first n smallest $M(S_i, S_j)$ values are selected to form a set of sequence pairs, denoted by I . Then, $U_3(P, \infty)$ is computed as:

$$U_3(P, \infty) = \min_{(S_i, S_j) \in I} \text{LCS}(S_i^P, S_j^P) \quad (7)$$

The time complexity for computing $U_3(P, \infty)$ is $O(n)$. Finally, a tighter upper bound $U(P, \infty) = \min \{U_1(P, \infty), U_2(P, \infty), U_3(P, \infty)\}$ is obtained with a time complexity $O(|\Sigma|n) + O(n) + O(n) = O(|\Sigma|n)$.

The A^* -type algorithms have the following defects:

- They must use a priority queue to store nodes, and the time complexity to maintain this priority queue is $O(\log(V))$, where V is the number of nodes.
- They obtain a tighter upper bound by estimating several upper bounds and then selecting the best one. Estimating several upper bounds needs more time and computation.
- In order to prevent redundant nodes in DAG, they still need to store the DAG they have built in the memory during the search, resulting in large memory consumption.

Therefore, the time and space costs of finding all exact MLCSs using the A^* -type algorithms are still large. To overcome these drawbacks, we propose a new algorithm.

3. A Distributed Storage MLCS Algorithm with Time Efficient Upper Bound and Precise Lower Bound: Step-MLCS

In this section, we first propose a time-efficient upper bound estimation method that uses less time to estimate the upper bound and then design a strategy to estimate a precise lower bound. Finally, we proposed a distributed storage scheme.

3.1. A Time Efficient Upper Bound Estimation Strategy

Both the Big-MLCS and A^* -type algorithms use all subsequences $\{S_1^P, S_2^P, \dots, S_n^P\}$ after node P when calculating $U(P, \infty)$, but it should be noted that two shorter subsequences in $\{S_1^P, S_2^P, \dots, S_n^P\}$ tend to have the shortest longest common subsequence (LCS). To use less time to estimate the upper bound $U(P, \infty)$, we select only θ shortest sequences among $\{S_1^P, S_2^P, \dots, S_n^P\}$, where θ is the threshold. Without loss of generality, let $\{S_1^P, S_2^P, \dots, S_\theta^P\}$ denote the θ shortest subsequences, and S_* denote the shortest subsequence among them. The details of the θ assignment are as follows.

- When $n \leq 10$, $\theta = \lceil 0.5 \cdot n \rceil$.
- When $10 < n \leq 100$, $\lceil 0.15 \cdot n \rceil \leq \theta < \lceil 0.5 \cdot n \rceil$ (i.e., θ is between 15% and 50% of n . The larger the number of sequences, the smaller the percentage should be). For example, when the number of sequences is 100, θ can be taken as $0.15 \cdot 100 = 15$.
- When $100 < n \leq 1000$, $\lceil 0.01 \cdot n \rceil \leq \theta < \lceil 0.15 \cdot n \rceil$.
- When $n > 1000$, $5 \leq \theta < 10$.

With threshold θ , we can quickly estimate our proposed upper bound $U_4(P, \infty)$ using Algorithm 1.

Algorithm 1: Time Efficient Upper Bound Estimation Strategy

Input: Match point $P = (p_1, p_2, \dots, p_n)$, S_* , $\{S_1^P, S_2^P, \dots, S_\theta^P\}$

1: **for** $i = 1$ to θ , $S_i^P \neq S_*$ **do**

2: Compute $\text{diversity}(S_*, S_i^P)$

3: **end for**

4: $U_4(P, \infty) \leftarrow$ the length of the LCS between S_* and S_k^P with largest $\text{diversity}(S_*, S_k^P)$

After obtaining LCS during preprocessing, the complexity of obtaining the largest $\text{diversity}(S_*, S_k^P)$ and $U_4(P, \infty)$ is $O(|\Sigma|\theta)$. It should be noted that θ is significantly smaller than n .

3.2. Construct Smaller DAG and Estimate Precise Lower Bound

The DAG construction and precise lower-bound estimation are executed in an alternative manner as follows:

First, we estimate the initial lower bound L_{MLCS} . Find an initial path using the method in Big-MLCS: choose the node with the minimum sum of components in each level and record this sum at each level (the sum in level k is denoted $\text{Sum}(k)$). These nodes at all levels form a path. Its length was used as the initial lower bound L_{MLCS} of the MLCS. For example, with three sequences,

$S_1 = ACAGTT$
 $S_2 = CAGGCT$
 $S_3 = GACAGTA$

$A(1, 2, 2)$, $C(2, 5, 3)$, and $T(5, 6, 6)$ in Fig. 1 are the nodes in levels 1 to 3, respectively, and the sums of their components are $Sum(1) = 5$, $Sum(2) = 10$, and $Sum(3) = 17$, respectively. These three nodes form a path of length 3, i.e., $L_{MLCS} = 3$.

Second, let $L(0) = \{O(0, 0, \dots, 0)\}$ denote the set of nodes at Level 0. Let the number of levels be $k = 0$. Put all successor nodes of all nodes of $L(k)$ into a set $Suc(k+1)$ and put the nodes of $Suc(k+1)$ with the minimal coordinate into a set $MinSuc(k+1)$. For a randomly chosen node, $P(p_1, p_2, \dots, p_n) \in MinSuc(k+1)$, we denote $Sum_{(k+1)}^P = \sum_{i=1}^n p_i$.

Algorithm 2: Construct Smaller DAG and Estimate Precise Lower Bound

Input: Sequences set $S = \{S_1, \dots, S_i, \dots, S_n\}$

```

1:  $L_{MLCS}$  and  $Sum(k) \leftarrow$  initial lower bound estimation
2:  $L(0) = \{O(0, 0, \dots, 0)\}$ 
3:  $Suc(1) \leftarrow$  successor nodes of  $O(0, 0, \dots, 0)$ 
4:  $MinSuc(1) \leftarrow$  nodes with the minimal coordinate in the  $Suc(1)$ 
5:  $k = 0$ 
6: for  $P \in MinSuc(k+1)$  and  $MinSuc(k+1) \neq \emptyset$  do
7:   if  $Sum_{(k+1)}^P < Sum(k+1)$  then
8:     update  $L_{MLCS}$  by always using only one of the two methods:
9:     Method 1:
10:    for  $Suc(P) \neq (\infty, \infty, \dots, \infty)$  do
11:       $P$  in next level  $\leftarrow$  node with the minimal sum of coordinates
12:    end for
13:    Method 2:
14:    for  $Suc(P) \neq (\infty, \infty, \dots, \infty)$  do
15:       $P$  in next level  $\leftarrow$  node with the maximal  $U_4(Q^*, \infty)$ 
16:    end for
17:    if the length of the path through all nodes  $> L_{MLCS}$  then
18:      update  $L_{MLCS}$  and each  $Sum(k)$ 
19:    end if
20:  else
21:     $L_{MLCS}$  and each  $Sum(k)$  is unchanged
22:  end if
23:   $U_4(P, \infty) \leftarrow$  estimate the upper bound of  $P$ 
24:  if  $U(O, P, \infty) \geq L_{MLCS}$  then
25:    put  $P$  in  $L(k+1)$ 
26:  end if
27:   $k = k + 1$ 
28:  $Suc(k+1) \leftarrow$  successor nodes of all nodes of  $L(k)$ 
29:  $MinSuc(k+1) \leftarrow$  nodes with the minimal coordinate in the  $Suc(k+1)$ 
30: end for

```

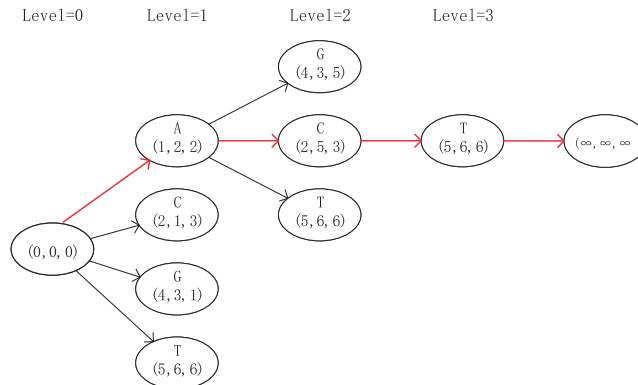


Fig. 1. Find a path as long as possible and use its length as the lower bound.

If $Sum_{(k+1)}^P < Sum(k+1)$, adaptively update lower bound L_{MLCS} by always using only one of the following two methods:

- (1) Take P as the current node in level $k+1$, find the set of all its successors, denoted by $Suc(P)$, and choose one in $Suc(P)$ with the minimal sum of coordinates as the next node P at level $k+2$. Record the minimal sum by $Sum_{(k+2)}^P$. This process is repeated until the current node has no successors. We then obtain a path through all the nodes P . If its length is greater than L_{MLCS} , replace L_{MLCS} by its length and $Sum(k)$ by $Sum_{(k)}^P$ for each level k ; otherwise, L_{MLCS} and $Sum(k)$ remain unchanged.
- (2) Take P as the current node in level $k+1$, and let $Suc(P)$ denote the set of all its successor nodes. For each node $Q \in Suc(P)$, compute the upper bound $U_4(Q, \infty)$ using the method in the previous subsection. Find the node Q^* with the maximal $U_4(Q^*, \infty)$ as node P at level $k+2$ and record the sum of its coordinates $Sum_{(k+2)}^P$. This process is repeated until the current node has no successors. We then obtain a path through all the nodes P . If its length is greater than L_{MLCS} , replace L_{MLCS} by its length and $Sum(k)$ by $Sum_{(k)}^P$ for each level k ; otherwise, L_{MLCS} and $Sum(k)$ remain unchanged.

If $Sum_{(k+1)}^P \geq Sum(k+1)$, then the lower bound L_{MLCS} remains unchanged. For $P \in MinSuc(k)$, we use a branch-and-bound scheme to judge whether P is put in $L(k+1)$ of the DAG as follows: estimate the upper bound $U_4(P, \infty)$. Let $U(O, P, \infty) = Level(P) + U_4(P, \infty)$. If $U(O, P, \infty) < L_{MLCS}$, then all paths through P are not the longest path, and we do not place P in $L(k+1)$ of the DAG. Otherwise, we set P in $L(k+1)$ of the DAG, and add an edge from P to its predecessor node. For node set $L(k+1)$ at level $k+1$, by repeating the above process on $L(k+1)$, we can obtain $L(k+2)$. Similarly, we can obtain node sets in later levels until a level M is found, such that any node in $L(M)$ has no successor. Place an edge from each node in $L(M)$ to the end node $(\infty, \infty, \dots, \infty)$. The pseudocode is shown in Algorithm 2, and an example of the process of Algorithm 2 is provided in Appendix C.

Through the precise lower-bound estimation strategy, we can adaptively improve L_{MLCS} while constructing the DAG. Therefore, we can construct a smaller DAG and reduce the time and space consumption.

3.3. Distributed Storage Scheme

Note that the internal computer memory has fast reading and writing speeds but a small storage capacity and is inconvenient for expansion. The external storage device has slow reading and writing speeds but a large storage capacity and is easy to expand. Therefore, it is unnecessary to store the entire DAG in the internal memory. Note that the DAG is expanded level by level, and after $L(k)$ in the DAG is constructed, it will not change in the subsequent levels of the DAG. Therefore, we can use a distributed storage strategy to store the previously constructed DAG on different external storage devices (e.g., a mobile hard disk, data server, or cloud storage resource).

The main idea is to sequentially divide the DAG into several parts. Each successive λ level forms one part, where λ is a given integer. Note that after the DAG is completely built from $L(1)$ to $L(\lambda)$ (i.e., the DAG in the first part), the DAG in the first part does not change later. Therefore, after we obtain the nodes in $L(\lambda+1)$ from $L(\lambda)$, the first part of the DAG can be stored in the external storage device to save memory space, and the memory only stores: 1) nodes in $L(\lambda)$ and 2) all predecessors of nodes in $L(\lambda+1)$. Similarly, after building the DAG from $L(\lambda+1)$ to $L(2\lambda)$ (i.e., the second part of the DAG) and obtaining the nodes in $L(2\lambda+1)$ from $L(2\lambda)$, the second part of the DAG can be stored in the external storage device. The memory stores only: 1) nodes in $L(2\lambda)$ and 2) all predecessors of nodes in $L(2\lambda+1)$. Similarly, we can store the latter part of the DAG in the external storage device. The memory stores only the nodes at the current level, and the predecessors of the nodes at the next level. All built parts can be stored in different storage devices. As a result, memory storage and time consumption are greatly reduced, and the predecessors of each node can be quickly found, and thus all MLCS can be quickly found. An example of our distributed storage scheme is provided in Appendix D.

In summary, the proposed distributed storage strategy has the advantage that only the nodes in two successive levels are stored in memory and the others are stored in a distributed manner in different external devices. This significantly reduces the memory consumption. As a result, the proposed algorithm can solve larger-scale MLCS problems more quickly than the existing state-of-the-art algorithms.

4. Experiments and Analysis

All experiments were executed on a workstation with the main hardware AMD 3700X CPU, 64G RAM, and a 1T Solid State Disk. The link for the code for the proposed algorithm and the datasets is <https://github.com/Wang-Chun-Yang/Step-MLCS>. The symbol “*” means that the algorithm cannot get the answer in the given 64G RAM, and “+” means that the algorithm cannot get the result in 3600 s.

First, we compared the quality of the upper bounds $U_1(P, \infty)$, $U_2(P, \infty)$ and $U_3(P, \infty)$ used in the A^* -type algorithms and our proposed upper bound $U_4(P, \infty)$ on dataset BL [20,21] with different combinations of n , m , and $|\Sigma|$, where n and m are the number and length of the sequences, respectively, and $|\Sigma|$ is the size of the alphabet. The results are presented in Table 1. For each instance with a given n , m , and $|\Sigma|$ in Table 1, for a fair comparison, we compare the quality of the four aforementioned

Table 1The Percentage of Best Upper Bounds Obtained by Each of $U_1(P, \infty)$, $U_2(P, \infty)$, $U_3(P, \infty)$ and $U_4(P, \infty)$.

n		$m = 100$			
		$U_1(P, \infty)$	$U_2(P, \infty)$	$U_3(P, \infty)$	$U_4(P, \infty)$
$ \Sigma = 4$	10	6.76%	33.48%	46.84%	80.65%
	50	49.91%	14.12%	29.75%	72.40%
	100	46.70%	8.07%	17.67%	64.52%
	150	48.16%	12.08%	14.13%	70.80%
	200	41.06%	6.84%	12.02%	74.47%
		$m = 500$			
$ \Sigma = 12$	10	0.02%	25.60%	23.41%	82.09%
	50	0.04%	11.81%	14.63%	86.82%
	100	0.04%	5.57%	8.21%	93.32%
	150	0.11%	6.14%	7.30%	92.98%
	200	0.06%	5.50%	7.50%	91.97%
		$m = 500$			
$ \Sigma = 20$	10	0.04%	25.81%	33.11%	85.54%
	50	0.08%	7.62%	7.75%	94.42%
	100	0.19%	4.29%	6.43%	95.84%
	150	0.16%	3.02%	5.30%	96.69%
	200	0.19%	2.29%	3.85%	97.66%

upper bounds until the DAG contains one million nodes (when $|\Sigma| = 12$ and $|\Sigma| = 20$, if $m = 100$, the scale of the constructed DAG by some instances is too small to reach one million nodes, so when $|\Sigma| = 12$ and $|\Sigma| = 20$, we choose the instances of $m = 500$). Table 1 lists the percentage of the best upper bounds obtained by each upper bound estimation method. Note that sometimes, the best upper bound may be obtained by more than one method for one instance. The experimental results show that our proposed upper bound $U_4(P, \infty)$ obtains most of the best upper bounds for each instance and performs better than $U_1(P, \infty)$, $U_2(P, \infty)$, and $U_3(P, \infty)$. Among the 15 instances, $U_4(P, \infty)$ gets more than 80% best results for 11 instances and 64.52% to 74.47% for only four instances, whereas $U_1(P, \infty)$, $U_2(P, \infty)$ and $U_3(P, \infty)$ only get at most 49.91%, 33.48%, and 46.84%, respectively. When $|\Sigma| = 4$, $U_1(P, \infty)$ performs better than $U_2(P, \infty)$ and $U_3(P, \infty)$, and $U_3(P, \infty)$ performs better than $U_2(P, \infty)$. When $|\Sigma| = 12$ and $|\Sigma| = 20$, $U_3(P, \infty)$ performs slightly better than $U_2(P, \infty)$ in most instances.

Second, we compare the time used to build the DAG and the total number of nodes in the built DAG in Table 2 when the upper bound $U(P, \infty) = \min\{U_1(P, \infty), U_2(P, \infty), U_3(P, \infty)\}$ used in A^* -type algorithms (Horn et al.[21]) and the proposed upper bound $U_4(P, \infty)$ are used in the proposed algorithm. It can be seen from Table 2 that for five instances with $|\Sigma| = 4$ and $m = 90$, the number of nodes of the DAG constructed using $U_4(P, \infty)$ is larger than that of the DAG constructed using $U(P, \infty) = \min\{U_1(P, \infty), U_2(P, \infty), U_3(P, \infty)\}$ for two instances, that is, the DAG constructed using $U_4(P, \infty)$ is smaller than that constructed using $U(P, \infty)$ for two instances. The DAG constructed using $U_4(P, \infty)$ was larger than that constructed using $U(P, \infty)$ for three instances. However, for all five instances, the proposed algorithm (using $U_4(P, \infty)$) requires significantly less time than the algorithms using $U(P, \infty) = \min\{U_1(P, \infty), U_2(P, \infty), U_3(P, \infty)\}$. In addition, when $|\Sigma| = 4$ and $m = 100$, using the upper bound $U(P, \infty)$ (Horn et al.[21]), the final result can be obtained for only one instance $n = 10$ among five instances in 3600 s. However, the proposed algorithm can obtain the final results for all five instances, and the time required is significantly less than 3600 s. Besides, we directly compare the average time used by A^* algorithm and our proposed Step-MLCS algorithm on the dataset BL when both algorithms get all optimal solutions within 3600 s. From Table 3, we can see that Step-MLCS can obtain results in limited time and space on all instances. However, for the instances with $|\Sigma| = 4$, except for the instances of $n = 10$, the A^* algorithm cannot get the result in the limited time. This demonstrates the advantage of Step-MLCS in solving the large-scale MLCS problem. For other instances, both A^* algorithm and Step-MLCS can get optimal solutions in a very short time, because the size of DAG in these instances are relatively small. Therefore, the time used by Step-MLCS to get the exact solution is shorter than that used by A^* algorithm because the A^* algorithm calculates multiple upper bounds in order to improve the accuracy of the $U(P, \infty)$.

Third, we compare the proposed algorithm with two state-of-the-art exact algorithms, Top-MLCS [14] and Big-MLCS [15] on Fasta datasets in the experiments. In the experiments, two types of the most commonly used experimental scenarios were used: 1) The number of sequences remained unchanged at four, and the length of the sequence was gradually increased from 100 to 1000. 2) The length of the sequences remained unchanged at 90, and the number of sequences was gradually increased from 5 to 2000. For each case of every scenario, we conduct ten independent runs. The result for each case is the average result of ten results in ten runs. In addition, the sequences used for each case by each algorithm were identical. The results for all cases are listed in Tables 4–6 for scenario 1 and in Tables 7–9 for scenario 2. For notation convenience, let $\mathcal{A}1$ and $\mathcal{A}2$ denote Top-MLCS and Big-MLCS, respectively. Let $\mathcal{A}3$ and $\mathcal{A}4$ denote the proposed algorithm with the lower bound updated by the first and second methods, respectively. For the first scenario, Table 4 and Table 5 show the running time and memory consumption of $\mathcal{A}1$ to $\mathcal{A}4$, respectively. It can be seen from these tables that as the length of the sequence increases, the time and space costs of all algorithms gradually increase. When the sequence length is less than 400, all algo-

Table 2

The Comparison of Time Used to Build DAG and Total Number of Nodes in DAG by Using Two Upper Bounds.

n		$\min \{U_1(P, \infty), U_2(P, \infty), U_3(P, \infty)\}$		$U_4(P, \infty)$	
		$m = 90$			
		total nodes	running time	total nodes	running time
$ \Sigma = 4$	10	24887397	211.4	19805888	112.1
	50	28783814	1644.6	32115554	493.4
	100	19123657	1944.4	23997122	517.6
	150	17385326	2205.3	20171004	654.1
	200	28783814	1490.2	11239786	373.5
		$m = 100$			
$ \Sigma = 4$	10	52250085	429.2	51077819	279.1
	50	+	+	123760153	1838.9
	100	+	+	111023383	2111.2
	150	+	+	75139170	2092.1
	200	+	+	27335470	891.3

Table 3The Comparison of the Average Time Used by A^* and Step-MLCS on the Dataset BL When Both Algorithms Get All Optimal Solutions Within 3600 s.

n		$m = 100$	
$ \Sigma $		A^* algorithm	Step-MLCS
10	4	429.2	279.1
	12	3.76	2.68
	20	0.18	0.13
50	4	+	1838.9
	12	0.705	0.658
	20	< 0.01	< 0.01
100	4	+	2111.2
	12	0.288	0.036
	20	< 0.01	< 0.01
150	4	+	2092.1
	12	0.102	0.018
	20	< 0.01	< 0.01
200	4	+	891.3
	12	0.081	0.014
	20	< 0.01	< 0.01

Table 4For Scenario 1, Comparison of Running Time of Algorithms $\mathcal{A}1$ to $\mathcal{A}4$.

m	DNA ($ \Sigma = 4$)			
	$\mathcal{A}1$	$\mathcal{A}2$	$\mathcal{A}3$	$\mathcal{A}4$
200	23.1	3.9	2.1	1.9
300	160.0	31.5	10.7	13.8
325	195.9	51.4	20.8	27.0
350	291.3	85.3	29.5	32.8
375	387.2	90.0	35.7	48.1
400	576.5	142.7	58.7	65.5
425	*	211.2	62.3	82.5
450	*	236.5	78.6	108.2
475	*	360.6	130.8	149.1
500	*	482.7	176.7	208.4
525	*	537.6	193.0	223.8
550	*	623.0	215.7	251.2
575	*	761.0	290.8	347.8
600	*	1145.5	376.1	468.1
650	*	1849.9	516.3	626.5
700	*	2937.9	764.9	867.2
750	*	*	1110.9	1291.3
800	*	*	1339.7	1641.4
850	*	*	1831.6	2069.4
900	*	*	2192.3	2784.4
950	*	*	2864.4	3456.1
1000	*	*	(+)3692.7	(+)4718.6

Table 5For Scenario 1, Comparison of Memory Consumption of Algorithms $\mathcal{A}1$ to $\mathcal{A}4$.

m	DNA ($ \Sigma = 4$)			
	$\mathcal{A}1$	$\mathcal{A}2$	$\mathcal{A}3$	$\mathcal{A}4$
200	1.743	0.185	0.028	0.020
300	14.481	1.082	0.053	0.063
325	18.935	1.789	0.082	0.106
350	36.821	2.495	0.112	0.112
375	40.546	2.623	0.118	0.160
400	46.709	4.186	0.129	0.179
425	*	5.579	0.160	0.200
450	*	6.166	0.216	0.250
475	*	8.536	0.234	0.311
500	*	11.747	0.336	0.397
525	*	12.104	0.365	0.443
550	*	14.223	0.400	0.482
575	*	16.139	0.448	0.714
600	*	25.593	0.524	0.745
650	*	34.659	0.844	0.891
700	*	48.518	0.950	1.344
750	*	*	1.197	1.547
800	*	*	1.525	1.777
850	*	*	1.708	2.685
900	*	*	1.767	2.909
950	*	*	2.354	3.165
1000	*	*	2.986	3.363

Table 6For Scenario 1, Comparison of Number of Nodes In DAG of Algorithms $\mathcal{A}1$ to $\mathcal{A}4$.

m	DNA ($ \Sigma = 4$)			
	$\mathcal{A}1$	$\mathcal{A}2$	$\mathcal{A}3$	$\mathcal{A}4$
200	5928036	1625977	1196704	1099830
300	36104115	9888574	5529033	6995724
325	48133540	15266729	9620507	12575791
350	68056236	24420521	13766081	15393108
375	91149069	25426282	16712702	21517206
400	120413964	36625780	25729253	28955484
425	*	52084620	27888597	35583767
450	*	56611105	34028737	46148799
475	*	81692143	53290556	62408194
500	*	108967414	73759603	85632998
525	*	111586971	81682084	93094346
550	*	132264547	89095498	104118435
575	*	158332565	118583847	120005382
600	*	231975271	152990488	187544125
650	*	336695363	204705880	248635589
700	*	468479250	302936703	339434495
750	*	*	428530398	489981174
800	*	*	511675412	617246192
850	*	*	694090644	772979375
900	*	*	8233038211	1023046346
950	*	*	1057279003	1234051721
1000	*	*	1329755725	1655219096

gorithms can obtain results within the limited time and memory, but Top-MLCS takes the largest time and memory, followed by Big-MLCS, and finally, by our proposed Step-MLCS ($\mathcal{A}3$ and $\mathcal{A}4$, respectively). When the sequence length is greater than 400, the memory overhead of the Top-MLCS exceeds this limit because of the lack of a reasonable strategy to reduce the search space. Big-MLCS exceeds the memory limit when the sequence length is greater than 700. At this time, our Step-MLCS can still obtain the answer in the given memory space. Until the sequence length increases to 1000, the running time of Step-MLCS ($\mathcal{A}3$ and $\mathcal{A}4$) slightly exceeds our time limit a little bit (by 92 s and 1118 s, respectively), but the memory is within the limit. Furthermore, the time required by our Step-MLCS is only approximately 25% to 30% of the Big-MLCS, and the memory consumed by the Step-MLCS is also much smaller than that consumed by the Top-MLCS and Big-MLCS. To further compare these algorithms, we compare the total number of nodes of the DAG constructed by the compared algorithms in Table 6. We can see that the number of nodes of the DAG used by Step-MLCS ($\mathcal{A}3$ and $\mathcal{A}4$) is significantly smaller than that used by Top-MLCS and Big-MLCS, and $\mathcal{A}3$ performs better than $\mathcal{A}4$ in most cases.

Table 7
For Scenario 2, Comparison of Running Time of Algorithms $\mathcal{A}1$ to $\mathcal{A}4$.

n	DNA ($ \Sigma = 4$)				
	$ LCS $	$\mathcal{A}1$	$\mathcal{A}2$	$\mathcal{A}3$	$\mathcal{A}4$
5	38	3.8	0.8	0.3	0.4
10	32	*	558.1	110.0	154.7
20	25	*	*	1176.9	1227.5
40	22	*	903.4	361.1	296.8
60	21	*	2583.5	877.3	880.5
100	19	*	1941.5	554.2	562.6
150	18	*	2373.5	662.4	681.4
400	16	*	1564.2	297.9	298.9
800	15	*	1164.9	208.6	211.5
1000	14	*	1013.5	197.3	200.8
1500	14	*	1095.8	205.9	220.3
2000	13	1160.3	759.4	160.7	161.1

Table 8
For Scenario 2, Comparison of Memory Consumption of Algorithms $\mathcal{A}1$ to $\mathcal{A}4$.

n	DNA ($ \Sigma = 4$)				
	$ LCS $	$\mathcal{A}1$	$\mathcal{A}2$	$\mathcal{A}3$	$\mathcal{A}4$
5	38	0.608	0.035	0.005	0.007
10	32	*	14.384	1.203	1.596
20	25	*	*	16.441	17.461
40	22	*	11.402	5.315	5.200
60	21	*	22.967	11.659	11.652
100	19	*	12.079	8.699	8.599
150	18	*	15.454	7.623	7.511
400	16	*	16.664	4.987	4.977
800	15	*	3.094	2.939	2.910
1000	14	*	2.598	2.468	2.379
1500	14	*	2.497	2.222	2.189
2000	13	46.000	1.445	1.415	1.413

Table 9
For Scenario 2, Comparison of Number of Nodes In DAG of Algorithms $\mathcal{A}1$ to $\mathcal{A}4$.

n	DNA ($ \Sigma = 4$)				
	$ LCS $	$\mathcal{A}1$	$\mathcal{A}2$	$\mathcal{A}3$	$\mathcal{A}4$
5	38	1222377	241543	100437	141411
10	32	*	63743624	19779339	27281890
20	25	*	*	146925124	162268719
40	22	*	24335467	21546390	17937624
60	21	*	49814410	46535736	46489764
100	19	*	25395207	25109578	24144469
150	18	*	2625767	25333449	24878237
400	16	*	5430673	5261686	5195622
800	15	*	1888052	1883978	18114431
1000	14	*	1500102	1483950	1463400
1500	14	*	868318	844932	832463
2000	13	1523654	437780	431604	423005

For the second scenario, Tables 6–8 show the running time, space consumption, and total number of nodes used to construct the DAG of the four compared algorithms. When we increased the number of sequences, the difficulty of the problems first increased and then decreased.

This is because, as the number of sequences increases, the size of the DAG first increases and then decreases. Before the number of sequences does not reach the critical value, an increase in the size of the DAG will lead to an increase in computation and memory, and thus an increase in problem difficulty. After the critical value, the size of the DAG decreases, which results in a decrease in computation and memory, and thus a decrease in the problem difficulty. The critical value for n was approximately 20 in the experiments, that is, the most difficult problem was the one with approximately 20 sequences. Note that only the proposed Step-MLCS can obtain the final result in the limited time and memory for all cases, whereas Big-MLCS cannot obtain the final result in the given time limit for $n = 20$ and Top-MLCS cannot obtain the final results in most cases. It

can be seen from Table 7 that our Step-MLCS uses a much shorter time than Big-MLCS and Top-MLCS. This is because of the use of the threshold of Step-MLCS in Section 3.1, which can greatly reduce the candidate sequences when estimating the upper bound. Thus, the superiority of the time consumption of our algorithm is more significant. Table 8 shows that Step-MLCS still consumes less memory space than Top-MLCS and Big-MLCS, but the difference between Big-MLCS and Step-MLCS gradually decreases as the number of sequences increases. The main reason for this is that as the number of sequences increases, the estimated value of the lower bound will be more precise in the initialization phase. Therefore, the improvement of our strategy for updating the lower bound is not as significant as that for the long-sequence problems in Scenario 1. It can be seen from Table 9 that the sizes of Step-MLCS ($\mathcal{N}3$ and $\mathcal{N}4$) are smaller than those of Top-MLCS and Big-MLCS, and the DAG built by $\mathcal{N}4$ is slightly smaller than that by $\mathcal{N}3$. This indicates that when the number of sequences is large, our second method of updating the lower bound can find a more precise lower bound earlier. A more precise lower bound is conducive to deleting more nodes, thereby reducing the size of the DAG.

5. Conclusion

In this study, we propose a distributed storage algorithm to solve large-scale MLCS problems. Through a time-efficient upper bound estimation method, we can estimate the upper bound of the path from a node to the endpoint with less time cost. By proposing an adaptive lower-bound estimation strategy, we can obtain a more precise lower bound, which can lead to a smaller DAG. In addition, the proposed distributed storage scheme stores most parts of the DAG in different external storage devices, resulting in less memory consumption. As a result, the proposed algorithm can solve large-scale MLCS problems with a lower memory cost and higher speed.

In the future, we will further investigate the following issues: 1) Improve the estimation method for the lower bound. The experimental results show that the lower bound is particularly critical for problems with long sequences. 2) Study the parallel acceleration scheme for the algorithm. Many parts of the proposed algorithm appear to be independent and can be executed in parallel.

CRedit authorship contribution statement

Chunyang Wang: Conceptualization, Methodology, Software, Writing – original draft. **Yuping Wang:** Supervision, Funding acquisition, Conceptualization, Formal analysis, Writing – review & editing. **Xiangjuan Wu:** Resources, Formal analysis. **Xiaofang Guo:** Investigation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (No. 62272367, No. 62102304), Joint Funds of the National Natural Science Foundation of China (No. U22A2098) and the Fundamental Research Funds for the Central Universities, and the Innovation Fund of Xidian University (No. JYS2013).

Appendix A. The detailed procedure of Top-MLCS

The main steps are illustrated by the following example, considering three sequences:

$$S_1 = ACAGTT$$

$$S_2 = CAGGCT$$

$$S_3 = GACAGTA$$

A.1. Construct ICSCG:

Put a start point $O(0, 0, 0)$, and then connect it to each of its successors $A(1, 2, 2)$, $C(2, 1, 3)$, $G(4, 3, 1)$ and $T(5, 6, 6)$, as shown in Fig. 2a. Put these n -dimensional successors into the hash table and find all successors of each of these successors. Check whether any new successors exist in the hash table. This requires a comparison of the new successors with the nodes in the hash table. If someone is not in the hash table, connect it to its predecessors and stored in the hash table (in this example, nodes $C(2, 5, 3)$, $G(4, 3, 5)$, and $A(3, 2, 4)$). If someone is in the hash table ($T(5, 6, 6)$ in this example), we only need to connect it to its predecessors ($A(1, 2, 2)$, $C(2, 1, 3)$, and $G(4, 3, 1)$), as shown in Fig. 2b. If a node does not have a successor, connect

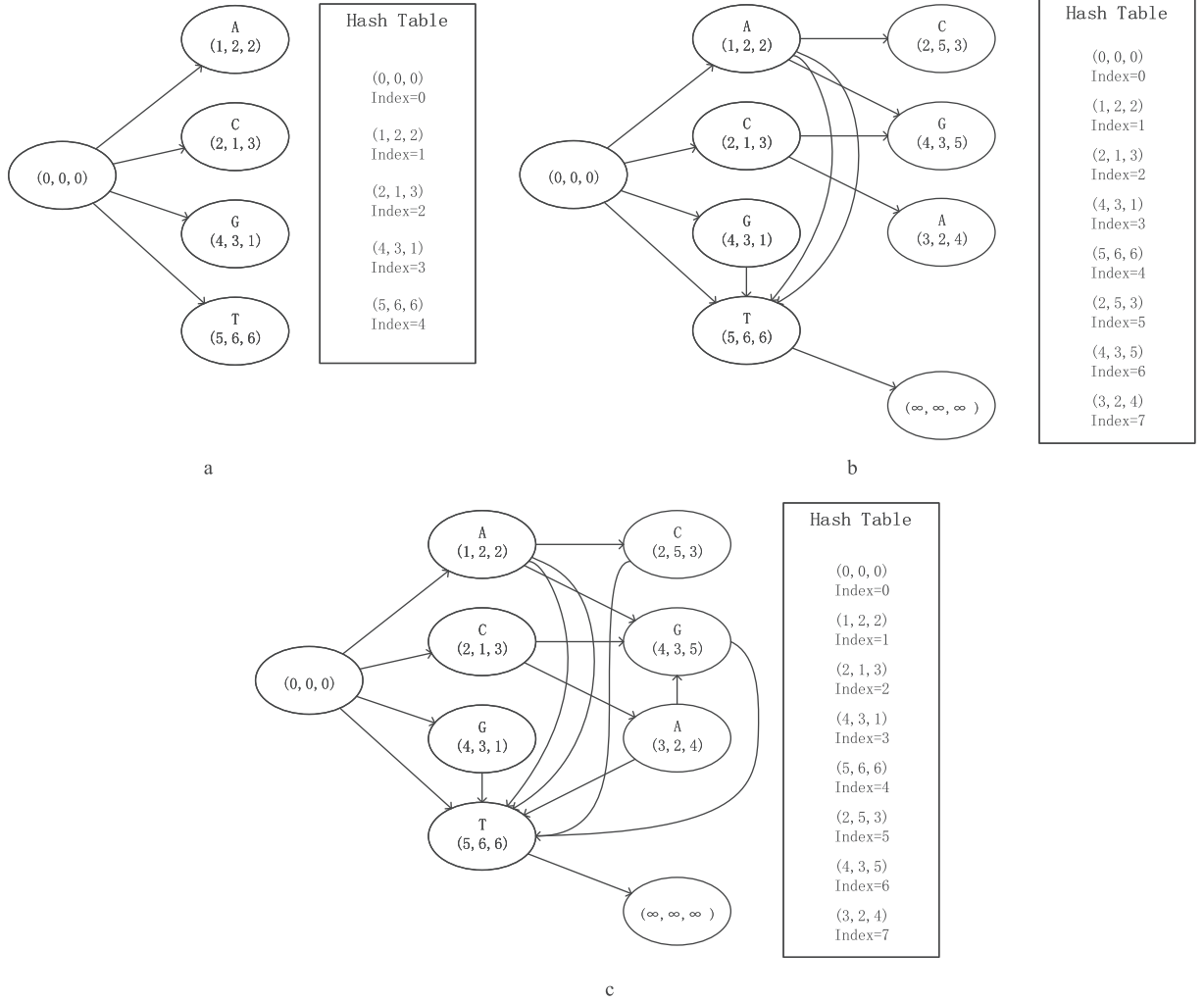


Fig. 2. Construct ICSG.

it directly to the endpoint. The above process is repeated until all the newly generated nodes in the graph have no successors. Then, we obtain the final ICSG and hash table that saves the n -dimensional coordinates of all nodes, as shown in Fig. 2c.

A.2. Determine Levels of Nodes by Forward Topological Sorting:

Forward topological sorting will start from the start node O . We mark its level as zero and delete the edges (dotted edges) from it, where the level is placed outside the circles, as shown in Fig. 3a. Then, we look for nodes with in-degree 0 and mark their level as 1, as shown in Fig. 3b. For nodes at level 1, the edges (dotted edges) are deleted. Then, we look for nodes with in-degree 0 and mark their level as 2, as shown in Fig. 3c. The above process is repeated for each node in Level 2 until the level of all points in the graph has been determined, as shown in Fig. 3d.

A.3. Backward Topological Sorting to Get the Final MLCS:

When we get the leveled graph by forward topological sorting (shown in Fig. 3d), we can use the backward topological sorting on the leveled graph to get the final graph as follows. If a path in the leveled graph has no nodes at a certain level, it must not be the longest path. This path could be deleted. After deleting all these paths (paths $O-A-C-\infty$ and $O-G-\infty$ in Fig. 3d), we obtain the final graph containing all the longest paths.

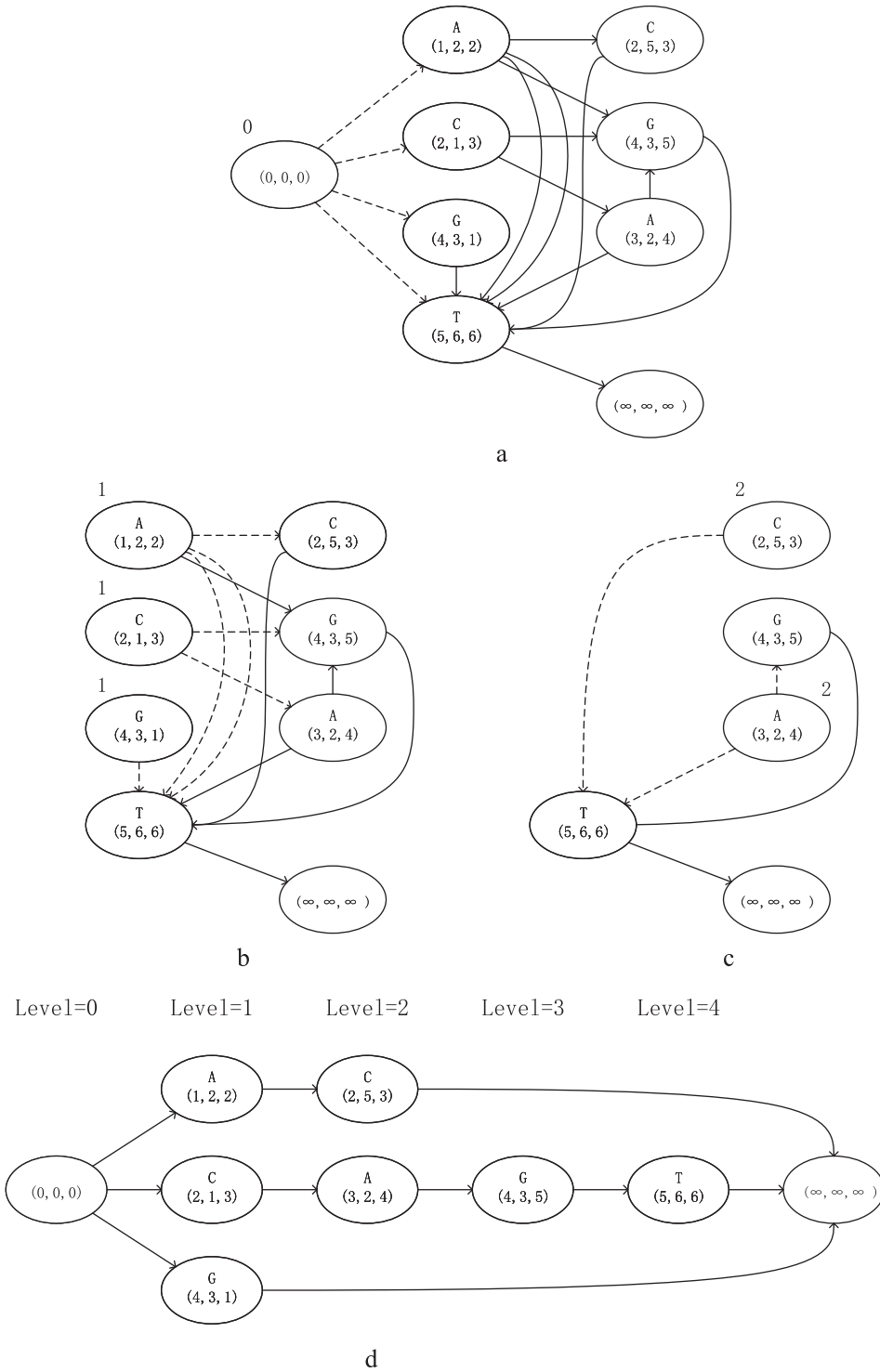


Fig. 3. Determine Levels of Nodes by Forward Topological Sorting.

Appendix B. An example on estimating upper bound and main steps of Big-MLCS

We assume that the key node is $A(1, 2, 2)$, and $Level(A(1, 2, 2)) = 1$. The subsequences after $A(1, 2, 2)$ are $S_1^P = CAGTT$, $S_2^P = GGCT$, and $S_3^P = CAGTA$. Because S_2^P is the shortest, it will be selected as S_* .

$$S_1 = ACAGTT$$

$$S_2 = CAGGCT$$

$$S_3 = GACAGTA$$

Compute $diversity(S_*, S_i^P)$ for $i = 1, 3$.

$$diversity(S_*, S_1^P) = \frac{0}{4}(0-1) + \frac{1}{4}(1-1) + \frac{2}{4}(2-1) + \frac{1}{4}(1-2) = \frac{1}{4}$$

$$diversity(S_*, S_3^P) = \frac{0}{4}(0-2) + \frac{1}{4}(1-1) + \frac{2}{4}(2-1) + \frac{1}{4}(1-1) = \frac{1}{2}$$

Because the diversity between S_3^P and S_* is the largest, the length of the LCS between S_3^P and S_* is 2, that is, the upper bound $U(A, \infty) = 2$. Thus, $U(O, A, \infty) = Level(A(1, 2, 2)) + U(A, \infty) = 1 + 2 = 3$,

If $U(O, P, \infty) < L_{MLCS}$, it means that all paths through P are not the longest path, and we will not further expand node P .

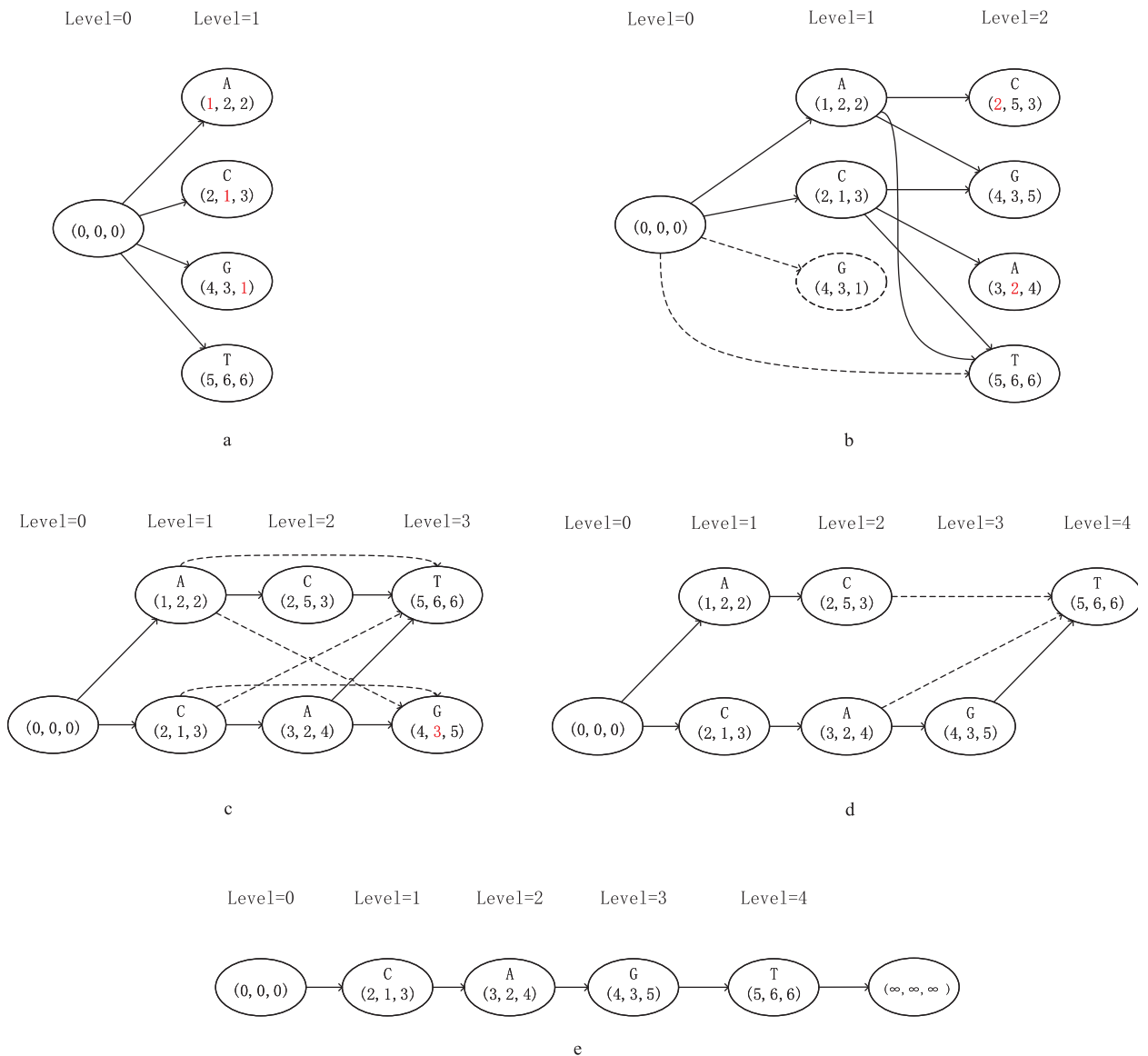


Fig. 4. Use branch-and-bound method to build DAG.

In this example, because $U(0, A, \infty) = L_{MLCS} = 3$, we cannot determine whether the paths passing through node $A(1, 2, 2)$ contain the longest path. We must extend the path from node $A(1, 2, 2)$ by connecting edges to its successor nodes $C(2, 5, 3)$, $G(4, 3, 5)$, and $T(5, 6, 6)$. Temporarily mark their level as 2, as shown in Fig. 4b. Thereafter, we randomly select a remaining node from $C(2, 1, 3)$ and $G(4, 3, 1)$ as a key node. Let's assume that we choose $C(2, 1, 3)$. Because $Level(C(2, 1, 3)) + U(C, \infty) = 4 > L_{MLCS} = 3$, we must extend the path from node $C(2, 1, 3)$, as shown in Fig. 4b. Now, there is only one remaining node $G(4, 3, 1)$ at level 1 and $Level(G(4, 3, 1)) + U(G, \infty) = 2 < L_{MLCS} = 3$. Therefore, no path through $G(4, 3, 1)$ can be the longest path. We deleted $G(4, 3, 1)$ from the graph and all paths from O to G (dotted edge), as shown in Fig. 4b. Similarly, we can extend the DAG to levels 3 and 4 and further to the end node, as shown in Fig. 4c, Fig. 4d, and Fig. 4e.

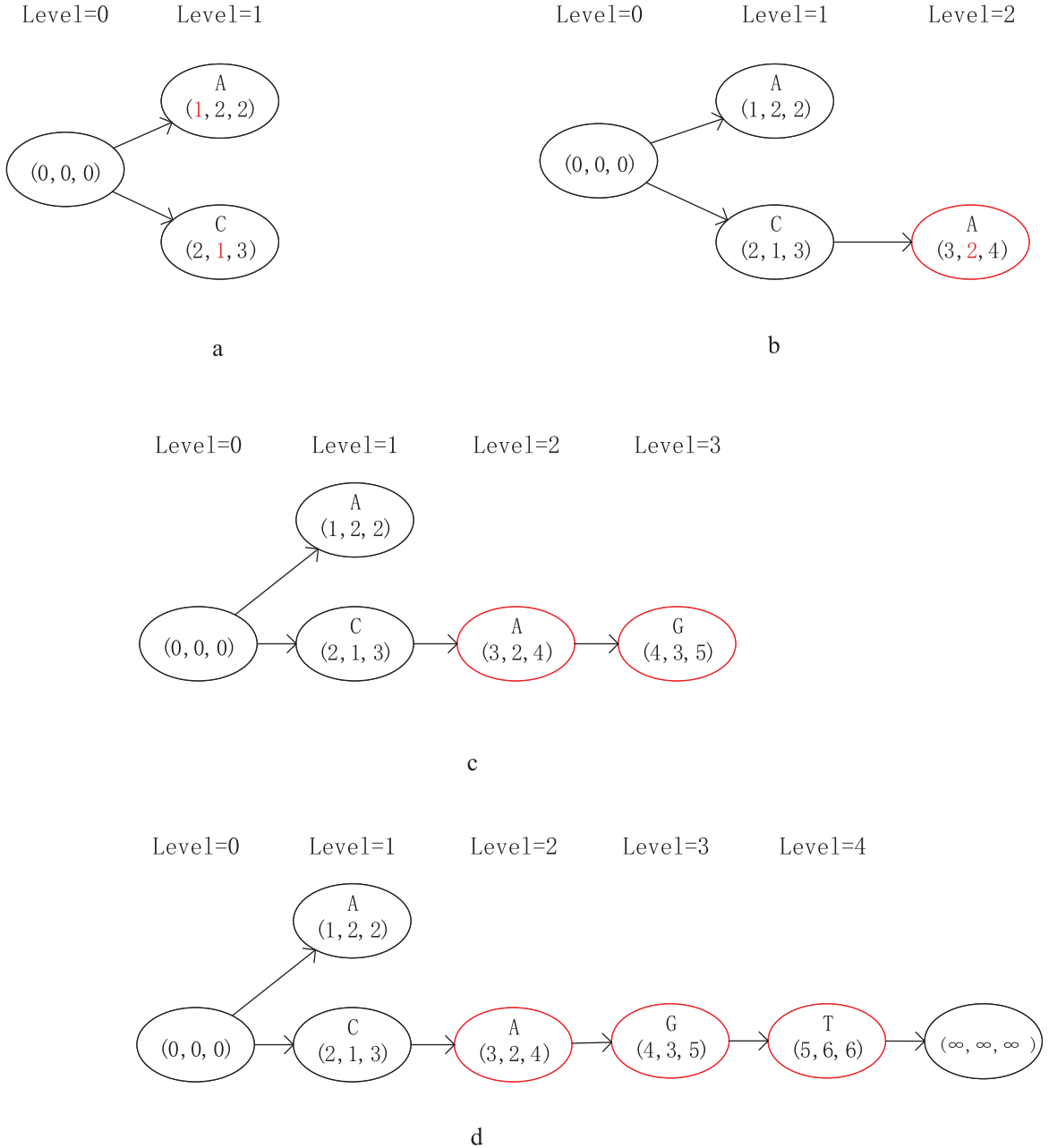


Fig. 5. Construct Smaller DAG and Estimate Precise Lower Bound.

Appendix C. An example of the process of Algorithm 2

For three sequences:

$$S_1 = ACAGTT$$

$$S_2 = CAGGCT$$

$$S_3 = GACAGTA$$

In this example, $Suc(1) = \{A(1, 2, 2), C(2, 1, 3), G(4, 3, 1), T(5, 6, 6)\}$, $MinSuc(1) = \{A(1, 2, 2), C(2, 1, 3), G(4, 3, 1)\}$. We use a branch-and-bound scheme to judge nodes in $MinSuc(1)$ and obtain $L(1) = \{A(1, 2, 2), C(2, 1, 3)\}$, as shown in Fig. 5a.

Similarly, we can get $Suc(2) = \{A(3, 2, 4), C(2, 5, 3), G(4, 3, 5), T(5, 6, 6)\}$, $MinSuc(2) = \{A(3, 2, 4), C(2, 5, 3)\}$. Because the sum of $A(3, 2, 4) = 9 < Sum(2) = 10$, we adaptively update the lower bound using either of the two methods proposed above and obtain $G(4, 3, 5)$ in level 3 and $T(5, 6, 6)$ in level 4, as shown in Fig. 5c and Fig. 5d. Because the length of this path is four,

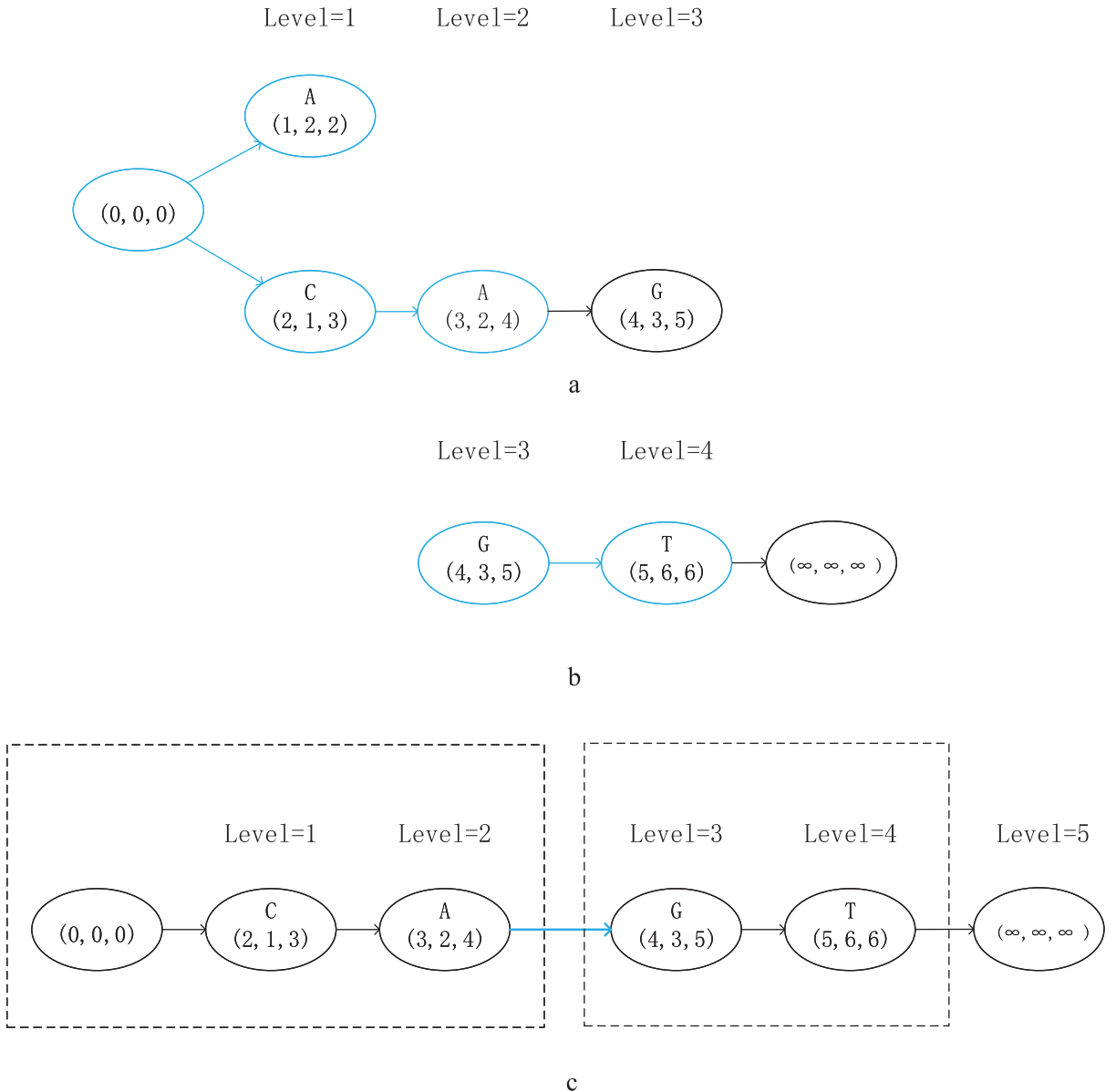


Fig. 6. Distributed Storage of DAG.

which is greater than the previous $L_{MLCS} = 3$, we update the lower bound to four. For another node $C(2, 5, 3)$ in $MinSuc(2)$, because $U(O, C, \infty) = 3$, it is less than our updated lower bound of $L_{MLCS} = 4$, we do not place it on the DAG and obtain $L(2) = \{A(3, 2, 4)\}$. By repeating this process, we obtain the final result.

Appendix D. An example of the proposed distributed storage scheme

We used the above example with three sequences to illustrate the distributed storage scheme for $\lambda = 2$.

$$S_1 = ACAGTT$$

$$S_2 = CAGGCT$$

$$S_3 = GACAGTA$$

After we construct the DAG from $L(1)$ to $L(2)$ and obtain $L(3)$ from $L(2)$, as shown in Fig. 6a, we store the DAG from $L(1)$ to $L(2)$ (marked in blue) in the external storage device, and the memory only stores: 1) nodes in $L(2)$ and 2) predecessors of nodes in $L(3)$. Similarly, after constructing the DAG from $L(3)$ to $L(4)$, we obtain the second part of the DAG and store it in an external storage device, as shown in Fig. 6b. The memory stores: 1) nodes in $L(4)$ and 2) the predecessors of nodes in $L(5)$. By using this scheme, we can know the predecessors of each node quickly and thus obtain all MLCSs quickly, as shown in Fig. 6c.

References

- [1] B. Nogrady, How cancer genomics is transforming diagnosis and treatment, *Nature* 579 (7800) (2020) S10–S11, <https://doi.org/10.1038/d41586-020-00845-4>.
- [2] L. Chaabane, Lamiche, A hybrid solver for protein multiple sequence alignment problem, *Journal of Bioinformatics and Computational Biology* 11 (1) (2018) 9111–9116, <https://doi.org/10.1142/S0219720018500154>.
- [3] B. Du, Z. Liu, F. Luo, Deep multi-scale attention network for RNA-binding proteins prediction, *Information Sciences* 582 (2022) 287–301, <https://doi.org/10.1016/j.ins.2021.09.025>.
- [4] E. Farhana, M.S. Rahman, Constrained sequence analysis algorithms in computational biology, *Information Sciences* 295 (2015) 247–257, <https://doi.org/10.1016/j.ins.2014.10.019>.
- [5] G. Agapito, P.H. Guzzi, M. Cannataro, Parallel and distributed association rule mining in life science: A novel parallel algorithm to mine genomics data, *Information Sciences* 575 (2021) 747–761, <https://doi.org/10.1016/j.ins.2018.07.055>.
- [6] X. Pan, H.-B. Shen, Scoring disease-microRNA associations by integrating disease hierarchy into graph convolutional networks, *Pattern Recognition* 105 (2020) 107385, ISSN 0031-3203, doi: 10.1016/j.patcog.2020.107385.
- [7] C. Zhang, Z. Du, W. Gan, P.S. Yu, TKUS: Mining top-k high utility sequential patterns, *Information Sciences* 570 (2021) 342–359, <https://doi.org/10.1016/j.ins.2021.04.035>.
- [8] J. Huang, Z. Fang, H. Kasai, LCS graph kernel based on Wasserstein distance in longest common subsequence metric space, *Signal Processing* 189 (2021), <https://doi.org/10.1016/j.sigpro.2021.108281>.
- [9] S. Hirschberg, Daniel, Algorithms for the longest common subsequence problem, *Journal of the Acm* 24 (1) (1977) 664–675, <https://doi.org/10.1145/322033.322044>.
- [10] J. Hunt, T. Szymanski, A Fast Algorithm for Computing Longest Common Subsequences, *Communications of the Acm* 20 (5) (1977) 350–353, <https://doi.org/10.1145/359581.359603>.
- [11] Y. Chen, A fast parallel algorithm for finding the longest common sequence of multiple biosequences, *Bmc Bioinformatics* 7 (Suppl 4) (2006) 1–12, <https://doi.org/10.1080/10556789808805713>.
- [12] P. Gustavsson, A. Syberfeldt, A New Algorithm Using the Non-Dominated Tree to Improve Non-Dominated Sorting, *Evolutionary Computation* 26 (1) (2018) 89–116, https://doi.org/10.1162/evco_a_00204.
- [13] Q. Wang, D. Korkin, Y. Shang, A Fast Multiple Longest Common Subsequence (MLCS) Algorithm, *IEEE Transactions on Knowledge and Data Engineering* 23 (3) (2011) 321–334, <https://doi.org/10.1109/TKDE.2010.123>.
- [14] Y. Li, Y. Wang, Z. Zhang, Y. Wang, D. Ma, J. Huang, A novel fast and memory efficient parallel MLCS algorithm for long and large-scale sequences alignments, in: 32nd IEEE International Conference on Data Engineering (ICDE), Helsinki, Finland, 1170–1181, doi: 10.1109/ICDE.2016.7498322, 2016.
- [15] C. Wang, Y. Wang, Y. Cheung, A branch and bound irredundant graph algorithm for large-scale MLCS problems, *Pattern Recognition* 119 (2021), <https://doi.org/10.1016/j.patcog.2021.108059>.
- [16] S. Liu, Y. Wang, W. Tong, S. Wei, A fast and memory efficient MLCS algorithm by character merging for DNA sequences alignment, *Bioinformatics* 36 (4) (2020) 1066–1073, <https://doi.org/10.1093/bioinformatics/btz725>.
- [17] S. Wei, Y. Wang, Y. Yang, S. Liu, A path recorder algorithm for Multiple Longest Common Subsequences (MLCS) problems, *Bioinformatics* 36 (10) (2020) 3035–3042, <https://doi.org/10.1093/bioinformatics/btaa134>.
- [18] J. Yang, Y. Xu, G. Sun, Y. Shang, A New Progressive Algorithm for a Multiple Longest Common Subsequences Problem and Its Efficient Parallelization, *IEEE Transactions on Parallel and Distributed Systems* 24 (5) (2013) 862–870, <https://doi.org/10.1109/TPDS.2012.202>.
- [19] J. Yang, Y. Xu, Y. Shang, G. Chen, A Space-Bounded Anytime Algorithm for the Multiple Longest Common Subsequence Problem, *IEEE Transactions on Knowledge and Data Engineering* 26 (11) (2014) 2599–2609, <https://doi.org/10.1109/TKDE.2014.2304464>.
- [20] M. Djukanovic, G.R. Raidl, C. Blum, Finding Longest Common Subsequences: New anytime A star search results, *Applied Soft Computing* 95 (2020), <https://doi.org/10.1016/j.asoc.2020.106499>.
- [21] M. Horn, G.R. Raidl, A*-Based Compilation of Relaxed Decision Diagrams for the Longest Common Subsequence Problem, in: P.J. Stuckey (Ed.), *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, Springer International Publishing, Cham, 2021, pp. 72–88, https://doi.org/10.1007/978-3-030-78230-6_5.